STAT 643:Nonparametric Statistics - Final Project

Capt Guadalupe Hernandez

2026-03-17

Load Data

Below we load the cloud seeding dataset.

```
cloud = read.table("cloud-seeding.txt",
                   header = TRUE,
                   sep = "\t",
                   col.names = c("unseeded", "seeded"))

kable(cloud, booktabs = TRUE) %>%
  kable_styling(latex_options = c("striped", "hold_position"), position = "center")
```

unseeded	seeded
1202.6	2745.6
830.1	1697.8
372.4	1656.0
345.5	978.0
321.2	703.4
244.3	489.1
163.0	430.0
147.8	334.1
95.0	302.8
87.0	274.7
81.2	274.7
68.5	255.0
47.3	242.5
41.1	200.7
36.6	198.6
29.0	129.6
28.6	119.0
26.3	118.3
26.1	115.3
24.4	92.4
21.7	40.6
17.3	32.7
11.5	31.4
4.9	17.5
4.9	7.7
1.0	4.1

Pivot the dataset, so that all the measurements appear in 1 column as opposed to 2 columns.

```
cloud_long = cloud %>%
  pivot_longer(cols = c(unseeded, seeded),
               names_to = "cloud_type",
               values_to = "rain_volume")

kable(cloud_long, booktabs = TRUE) %>%
  kable_styling(latex_options = c("striped", "hold_position"), position = "center")
```

cloud_type	rain_volume
unseeded	1202.6
seeded	2745.6
unseeded	830.1
seeded	1697.8
unseeded	372.4
seeded	1656.0
unseeded	345.5
seeded	978.0
unseeded	321.2
seeded	703.4
unseeded	244.3
seeded	489.1
unseeded	163.0
seeded	430.0
unseeded	147.8
seeded	334.1
unseeded	95.0
seeded	302.8
unseeded	87.0
seeded	274.7
unseeded	81.2
seeded	274.7
unseeded	68.5
seeded	255.0
unseeded	47.3
seeded	242.5
unseeded	41.1
seeded	200.7
unseeded	36.6
seeded	198.6
unseeded	29.0
seeded	129.6
unseeded	28.6
seeded	119.0
unseeded	26.3
seeded	118.3
unseeded	26.1
seeded	115.3
unseeded	24.4
seeded	92.4

unseeded	21.7
seeded	40.6
unseeded	17.3
seeded	32.7
unseeded	11.5
seeded	31.4
unseeded	4.9
seeded	17.5
unseeded	4.9
seeded	7.7
unseeded	1.0
seeded	4.1

Exploratory Data Analysis

Below we quickly list some summary statistics for our time t dataset.

```
cloud_long %>%
  group_by(cloud_type) %>%
  summarise(n      = n(),
            min    = min(rain_volume),
            q25    = quantile(rain_volume, 0.25),
            mean   = mean(rain_volume),
            median = median(rain_volume),
            q75    = quantile(rain_volume, 0.75),
            max    = max(rain_volume),
            sd     = sd(rain_volume),
            IQR    = q75 - q25)  %>%
  ungroup()
```

```
## # A tibble: 2 x 10
##   cloud_type     n   min   q25  mean median   q75   max    sd   IQR
##   <chr>      <int> <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 seeded        26   4.1  98.1  442.  222.   406. 2746.  651.  308.
## 2 unseeded      26   1    24.8  165.   44.2  159. 1203.  278.  134.
```

and now on the log10 transfromted scale:

```
cloud_long %>%
  group_by(cloud_type) %>%
  summarise(n      = n(),
            min    = min(log10(rain_volume)),
            q25    = quantile(log10(rain_volume), 0.25),
            mean   = mean(log10(rain_volume)),
            median = median(log10(rain_volume)),
            q75    = quantile(log10(rain_volume), 0.75),
            max    = max(log10(rain_volume)),
            sd     = sd(log10(rain_volume)),
            IQR    = q75 - q25)  %>%
  ungroup()
```

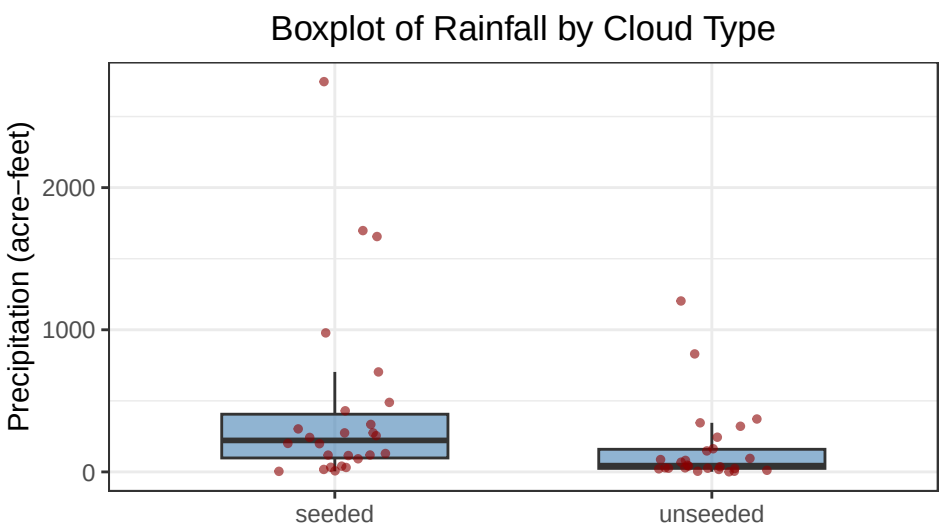
```
## # A tibble: 2 x 10
##   cloud_type     n   min   q25  mean median   q75   max    sd   IQR
##   <chr>      <int> <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 seeded        26 0.613  1.99  2.23   2.34  2.61  3.44 0.695 0.616
```

```
## 2 unseeded      26 0      1.39  1.73   1.64  2.20  3.08 0.713 0.807
```

Boxplot

Below we generate a boxplot of the dataset.

```
box_plot = ggplot(cloud_long, aes(x = cloud_type, y = rain_volume)) +
  geom_boxplot(width = 0.6, alpha = 0.6, fill = "steelblue", outlier.shape = NA) +
  geom_jitter(width = 0.15, alpha = 0.6, color = "darkred", size = 1) +
  labs(title = "Boxplot of Rainfall by Cloud Type",
       x = "",
       y = "Precipitation (acre-feet)") +
  theme_bw() +
  theme(plot.title = element_text(hjust = 0.5))

box_plot
```




```
# Save the plot as a PNG
ggsave(filename = "figures/box_plot.png",
  plot = box_plot,
  width = 5,
  height = 3,
  units = "in",
  dpi = 300)
```

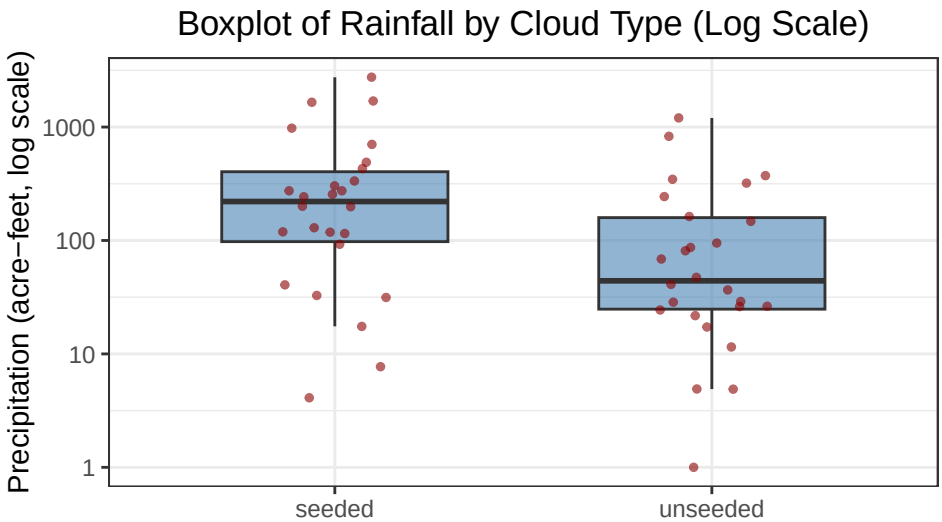
Our dataset exhibits severe right-skew.

Boxplot, Log Scale

Now we generate the boxplot for the data after it gets $\log_{10}$ transformed.

```
box_plot_log = ggplot(cloud_long, aes(x = cloud_type, y = rain_volume)) +
  geom_boxplot(width = 0.6, alpha = 0.6, fill = "steelblue", outlier.shape = NA) +
  geom_jitter(width = 0.15, alpha = 0.6, color = "darkred", size = 1) +
  scale_y_log10() +
  labs(title = "Boxplot of Rainfall by Cloud Type (Log Scale)",
       x = "",
       y = "Precipitation (acre-feet, log scale)") +
  theme_bw() +
```

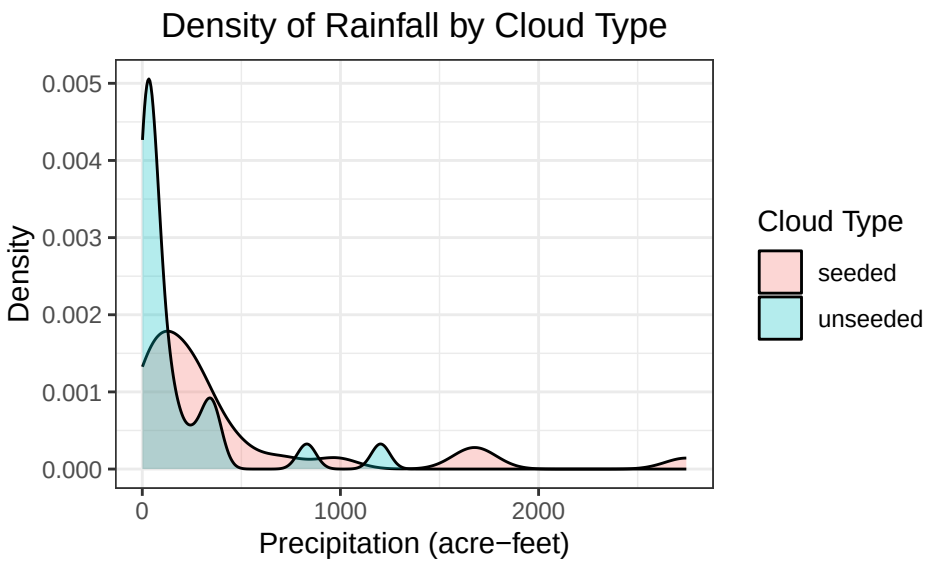
```
  theme(plot.title = element_text(hjust = 0.5))

box_plot_log
```




```
# Save the plot as a PNG
ggsave(filename = "figures/box_plot_log.png",
  plot = box_plot_log,
  width = 5,
  height = 3,
  units = "in",
  dpi = 300)
```

Density Plot

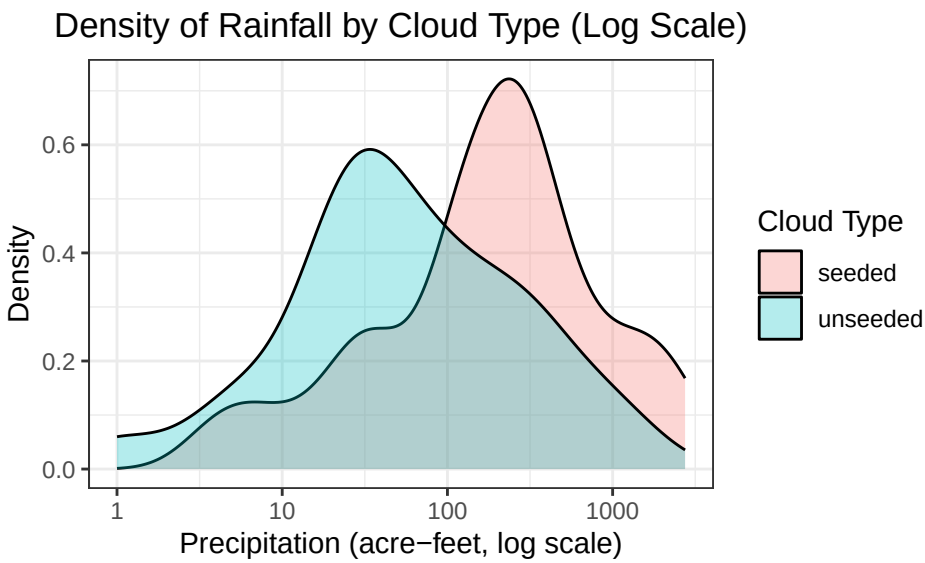
```
density_plot = ggplot(cloud_long, aes(x = rain_volume, fill = cloud_type)) +
  geom_density(alpha = 0.3) +
  labs(title = "Density of Rainfall by Cloud Type",
       x = "Precipitation (acre-feet)",
       y = "Density",
       fill = "Cloud Type") +
  theme_bw() +
  theme(plot.title = element_text(hjust = 0.5))

density_plot
```




```r
# Save the plot as a PNG
ggsave(filename = "figures/density_plot.png",
  plot = density_plot,
  width = 5,
  height = 3,
  units = "in",
  dpi = 300)
```

Density Plot, Log Scale

```r
density_plot_log = ggplot(cloud_long, aes(x = rain_volume, fill = cloud_type)) +
  geom_density(alpha = 0.3) +
  scale_x_log10() +
  labs(title = "Density of Rainfall by Cloud Type (Log Scale)",
       x = "Precipitation (acre-feet, log scale)",
       y = "Density",
       fill = "Cloud Type") +
  theme_bw() +
  theme(plot.title = element_text(hjust = 0.5))

density_plot_log
```




```r
# Save the plot as a PNG
ggsave(filename = "figures/density_plot_log.png",
  plot = density_plot_log,
  width = 5,
  height = 3,
  units = "in",
  dpi = 300)
```

Histogram, Seeded Data

We now want to visualize the distribution of the data using a histogram. We will select the optimal bandwidth according to the cross validation estimator of risk. The CV risk provides an approximation to the Integrated Mean Squared Error (IMSE) for histograms and can be expressed as

$$\hat{J}(h) = \frac{2}{h(n-1)} - \frac{n+1}{h(n-1)} \sum_{i=1}^{m} \hat{p}_j^2,$$

where $\hat{p}_j = \frac{Y_j}{n}$ is the proportion of data points in the jth bin, Y_j is the count in that bin, n is the sample size, h is the bin width, and m is the number of bins.

We compare four standard rules for determining the histogram bin width:

1. Freedman and Diaconis (1981)

$$h_{\text{FD}} = 2 \cdot IQR \cdot n^{-1/3}, \quad m_{\text{FD}} = \frac{(x_{(n)} - x_{(1)})}{h_{\text{FD}}}.$$

2. Scott's rule

$$h_{\text{Scott}} = \frac{3.5\hat{\sigma}}{n^{1/3}}, \quad m_{\text{Scott}} = \frac{(x_{(n)} - x_{(1)})}{h_{\text{Scott}}}.$$

3. Sturges' rule

$$m_{\text{Sturges}} = \log_2 n + 1, \quad h_{\text{Sturges}} = \frac{(x_{(n)} - x_{(1)})}{m_{\text{Sturges}}}.$$

4. Normal Reference Rule

$$h_n = \frac{1.06 \cdot \min\left\{s, \frac{\text{IQR}}{1.34}\right\}}{n^{1/5}}, \quad m_n = \frac{(x_{(n)} - x_{(1)})}{h_n}.$$

We compute the CV risk for each candidate bandwidth using the following function:

```
cv_risk_hist = function(x, h) {
  n = length(x)

  # Define bins explicitly so they depend on h
  breaks = seq(min(x), max(x) + h, by = h)

  # Histogram counts
  Y = hist(x, breaks = breaks, plot = FALSE)$counts

  # Empirical bin probabilities
  p_hat = Y / n

  # CV risk
  J_hat = (2 / (h * (n - 1))) - ((n + 1) / (h * (n - 1))) * sum(p_hat^2)

  return(J_hat)
}
```

Below you will find the code for the candidate bandwidths.

```
n = length(cloud$seeded)

h_fd = 2 * IQR(cloud$seeded) / n^(1/3)                # Freedman-Diaconis
h_scott = 3.5 * sd(cloud$seeded) / n^(1/3)            # Scott
h_sturges = diff(range(cloud$seeded)) / (log2(n) + 1) # Sturges
h_n = 1.06 * min(sd(cloud$seeded), IQR(cloud$seeded)/1.34) / n^(1/5)  # normal reference

h_candidates = c(FD = h_fd,
                 Scott = h_scott,
                 Sturges = h_sturges,
                 NormalRef = h_n)
```

We can then print the results using:

```
cv_results = data.frame(rule = names(h_candidates),
                        h = h_candidates,
                        cv_risk = sapply(X = h_candidates, FUN = cv_risk_hist, x = cloud$seeded))

cv_results
```

```
##                rule           h          cv_risk
## FD               FD 207.8652548 -0.0012599195157
## Scott         Scott 768.8635011 -0.0009141287572
## Sturges     Sturges 480.9278118 -0.0011956671494
## NormalRef NormalRef 126.9451421 -0.0014337833749
```

```
cv_results[which.min(cv_results$cv_risk), ]
```

```
##                rule           h         cv_risk
## NormalRef NormalRef 126.9451421 -0.001433783375
```

Based on the results, the Normal Reference rule produces the smallest CV risk. The optimal number of bins under the Normal Reference rule is

$$m_n = \frac{(x_{(n)} - x_{(1)})}{h_n} = \frac{(2745.6 - 4.1)}{126.9451} = 21.5959 \approx 22.$$

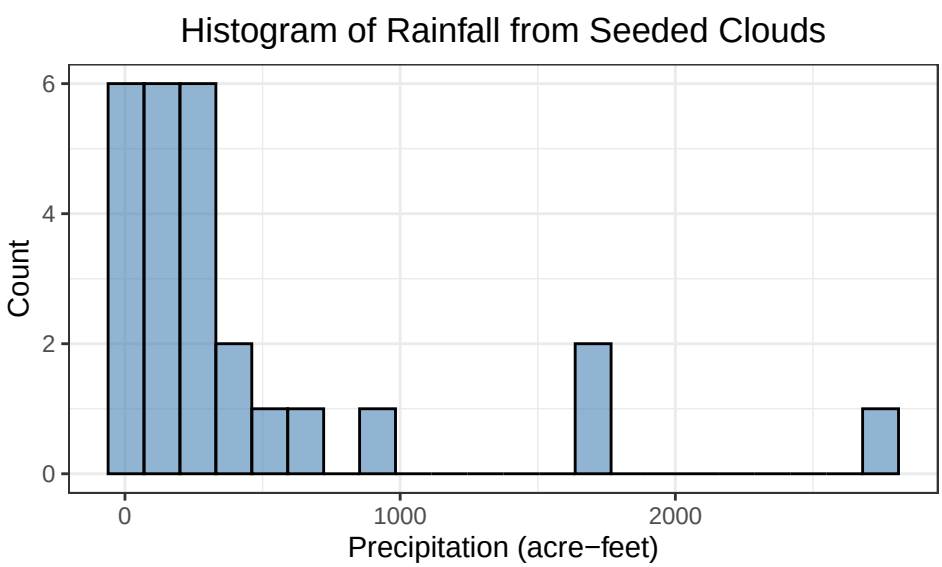
```r
m_n = ceiling( diff(range(cloud$seeded)) / h_n )
m_n
```

```
## [1] 22
```

The Normal Reference Rule formula gives a non-integer number of bins. Since a histogram requires an integer number of bins, we round up to the closest integer, 22.

Using the number of bins, we construct the histogram for the seeded cloud data.

```r
seeded_hist = ggplot(cloud, aes(x = seeded)) +
  geom_histogram(bins = m_n,
                 fill = "steelblue",
                 color = "black",
                 alpha = 0.6) +
  labs(title = "Histogram of Rainfall from Seeded Clouds",
       x = "Precipitation (acre-feet)",
       y = "Count") +
  theme_bw() +
  theme(plot.title = element_text(hjust = 0.5))

seeded_hist
```




```r
# Save the plot as a PNG
ggsave(filename = "figures/seeded_hist.png",
  plot = seeded_hist,
  width = 5,
  height = 3,
  units = "in",
  dpi = 300)
```

Histogram, Unseeded Data

We compute the CV risk for each candidate bandwidth using the following function:

```r
cv_risk_hist = function(x, h) {
  n = length(x)
```

```r
  # Define bins explicitly so they depend on h
  breaks = seq(min(x), max(x) + h, by = h)

  # Histogram counts
  Y = hist(x, breaks = breaks, plot = FALSE)$counts

  # Empirical bin probabilities
  p_hat = Y / n

  # CV risk
  J_hat = (2 / (h * (n - 1))) - ((n + 1) / (h * (n - 1))) * sum(p_hat^2)

  return(J_hat)
}
```

Below you will find the code for the candidate bandwidths.

```r
n = length(cloud$unseeded)

h_fd = 2 * IQR(cloud$unseeded) / n^(1/3)                # Freedman-Diaconis
h_scott = 3.5 * sd(cloud$unseeded) / n^(1/3)            # Scott
h_sturges = diff(range(cloud$unseeded)) / (log2(n) + 1) # Sturges
h_n = 1.06 * min(sd(cloud$unseeded), IQR(cloud$unseeded)/1.34) / n^(1/5)  # normal reference

h_candidates = c(FD = h_fd,
                 Scott = h_scott,
                 Sturges = h_sturges,
                 NormalRef = h_n)
```

We can then print the results using:

```r
cv_results = data.frame(rule = names(h_candidates),
                        h = h_candidates,
                        cv_risk = sapply(X = h_candidates, FUN = cv_risk_hist, x = cloud$unseeded))

cv_results
```

```
##                rule            h         cv_risk
## FD               FD  90.71741997 -0.004507135893
## Scott         Scott 328.94302401 -0.002136662538
## Sturges     Sturges 210.79075640 -0.002788597854
## NormalRef NormalRef  55.40192749 -0.005015526265
```

```r
cv_results[which.min(cv_results$cv_risk), ]
```

```
##                rule           h         cv_risk
## NormalRef NormalRef 55.40192749 -0.005015526265
```

Based on the results, the Normal Reference rule produces the smallest CV risk. The optimal number of bins under the Normal Reference rule is

$$m_n = \frac{(x_{(n)} - x_{(1)})}{h_n} = \frac{(1202.6 - 1.0)}{55.4019} = 21.6888 \approx 22.$$

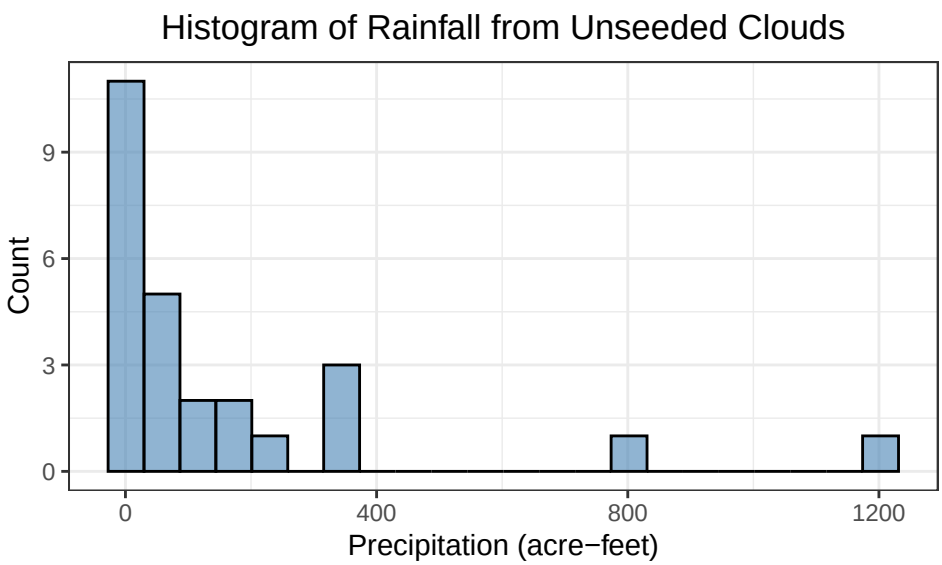
```r
m_n = ceiling( diff(range(cloud$unseeded)) / h_n )
m_n
```

```
## [1] 22
```

The Normal Reference Rule formula gives a non-integer number of bins. Since a histogram requires an integer number of bins, we round up to the closest integer, 22.

Using the number of bins, we construct the histogram for the seeded cloud data.

```
unseeded_hist = ggplot(cloud, aes(x = unseeded)) +
  geom_histogram(bins = m_n,
                 fill = "steelblue",
                 color = "black",
                 alpha = 0.6) +
  labs(title = "Histogram of Rainfall from Unseeded Clouds",
       x = "Precipitation (acre-feet)",
       y = "Count") +
  theme_bw() +
  theme(plot.title = element_text(hjust = 0.5))

unseeded_hist
```

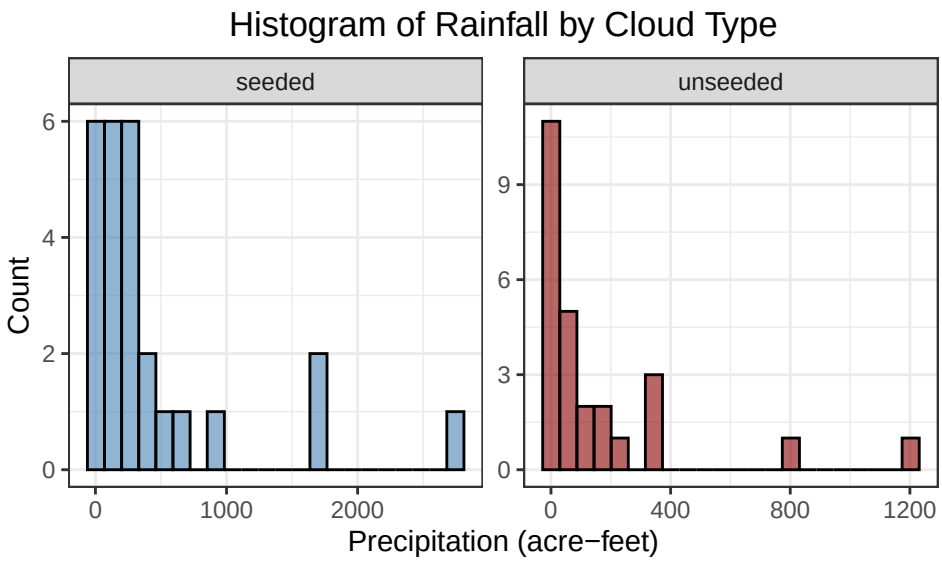



```
# Save the plot as a PNG
ggsave(filename = "figures/unseeded_hist.png",
  plot = unseeded_hist,
  width = 5,
  height = 3,
  units = "in",
  dpi = 300)
```

Combined Histogram

```
combined_hist = ggplot(cloud_long, aes(x = rain_volume, fill = cloud_type)) +
  geom_histogram(bins = 22,
                 color = "black",
                 alpha = 0.6) +
  scale_fill_manual(values = c("seeded" = "steelblue", "unseeded" = "darkred")) +
  facet_wrap(~ cloud_type, scales = "free") +
  labs(title = "Histogram of Rainfall by Cloud Type",
       x = "Precipitation (acre-feet)",
       y = "Count") +
```

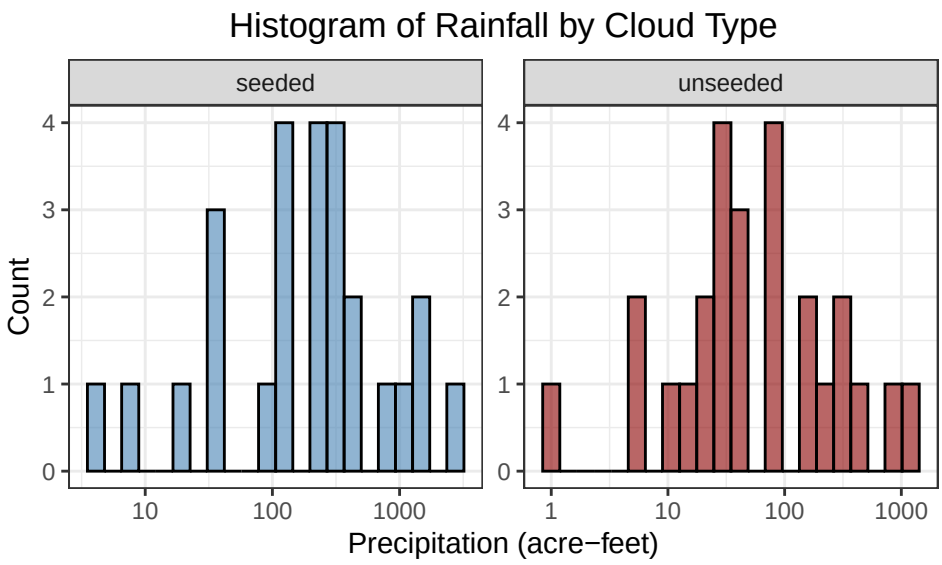
```
  theme_bw() +
  theme(plot.title = element_text(hjust = 0.5),
        legend.position = "none")

combined_hist
```




```
# Save the plot as a PNG
ggsave(filename = "figures/combined_hist.png",
  plot = combined_hist,
  width = 5,
  height = 3,
  units = "in",
  dpi = 300)
```

Combined Histogram Log Scale

```
combined_hist_log = ggplot(cloud_long, aes(x = rain_volume, fill = cloud_type)) +
  geom_histogram(bins = 22,
                 color = "black",
                 alpha = 0.6) +
  scale_fill_manual(values = c("seeded" = "steelblue", "unseeded" = "darkred")) +
  facet_wrap(~ cloud_type, scales = "free") +
  scale_x_log10() +
  labs(title = "Histogram of Rainfall by Cloud Type",
       x = "Precipitation (acre-feet)",
       y = "Count") +
  theme_bw() +
  theme(plot.title = element_text(hjust = 0.5),
        legend.position = "none")

combined_hist_log
```

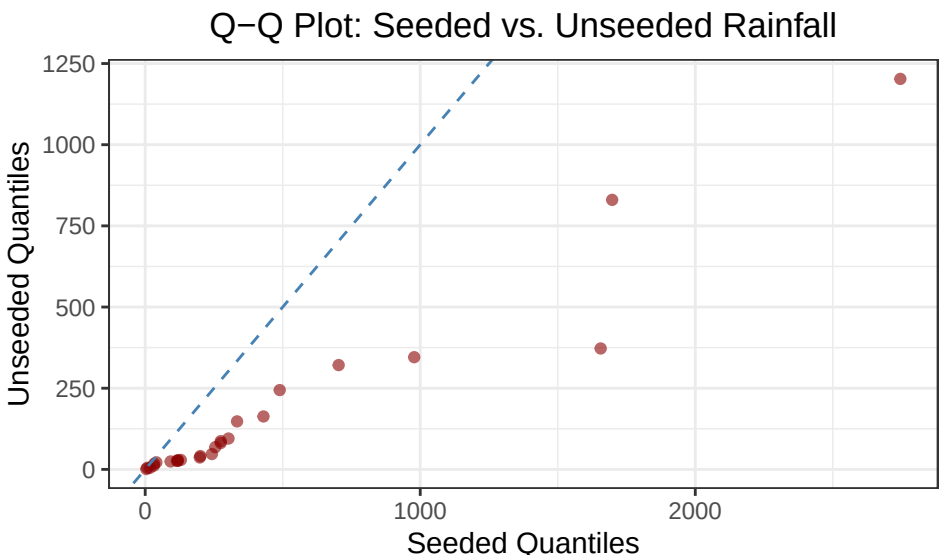



```
# Save the plot as a PNG
ggsave(filename = "figures/combined_hist_log.png",
  plot = combined_hist_log,
  width = 5,
  height = 3,
  units = "in",
  dpi = 300)
```

QQ Plot

Before conducting any formal tests, a normal QQ plot was generated to visually assess normality.

```
# Sort the data
seeded = sort(cloud$seeded)
unseeded = sort(cloud$unseeded)

# Make a data frame
qq_df = data.frame(seeded = seeded,
                   unseeded = unseeded)

qq_plot = ggplot(qq_df, aes(x = seeded, y = unseeded)) +
  geom_point(color = "darkred", size = 1.5, alpha = 0.6) +
  geom_abline(intercept = 0, slope = 1, linetype = "dashed", color = "steelblue") +
  labs(title = "Q-Q Plot: Seeded vs. Unseeded Rainfall",
       x = "Seeded Quantiles",
       y = "Unseeded Quantiles") +
  theme_bw() +
  theme(plot.title = element_text(hjust = 0.5))

qq_plot
```

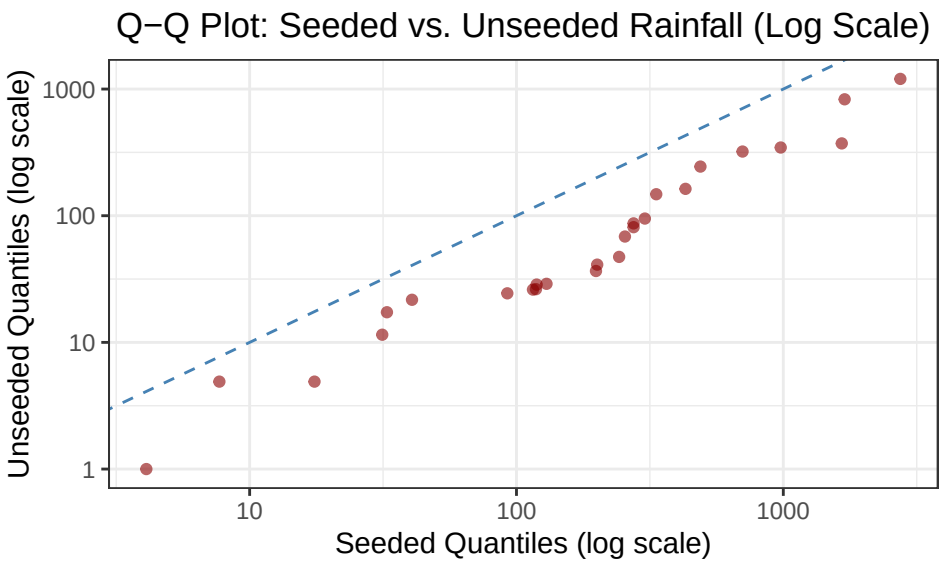



```
# Save the plot as a PNG
ggsave(filename = "figures/qq_plot.png",
  plot = qq_plot,
  width = 5,
  height = 3,
  units = "in",
  dpi = 300)
```

QQ Plot, Log Scale

Before conducting any formal tests, a normal QQ plot was generated to visually assess normality.

```
# Sort the data
seeded = sort(cloud$seeded)
unseeded = sort(cloud$unseeded)

# Make a data frame
qq_df = data.frame(seeded = seeded,
                   unseeded = unseeded)

qq_plot_log = ggplot(qq_df, aes(x = seeded, y = unseeded)) +
  geom_point(color = "darkred", size = 1.5, alpha = 0.6) +
  geom_abline(intercept = 0, slope = 1, linetype = "dashed", color = "steelblue") +
  scale_x_log10() +
  scale_y_log10() +
  labs(title = "Q-Q Plot: Seeded vs. Unseeded Rainfall (Log Scale)",
       x = "Seeded Quantiles (log scale)",
       y = "Unseeded Quantiles (log scale)") +
  theme_bw() +
  theme(plot.title = element_text(hjust = 0.5))

qq_plot_log
```



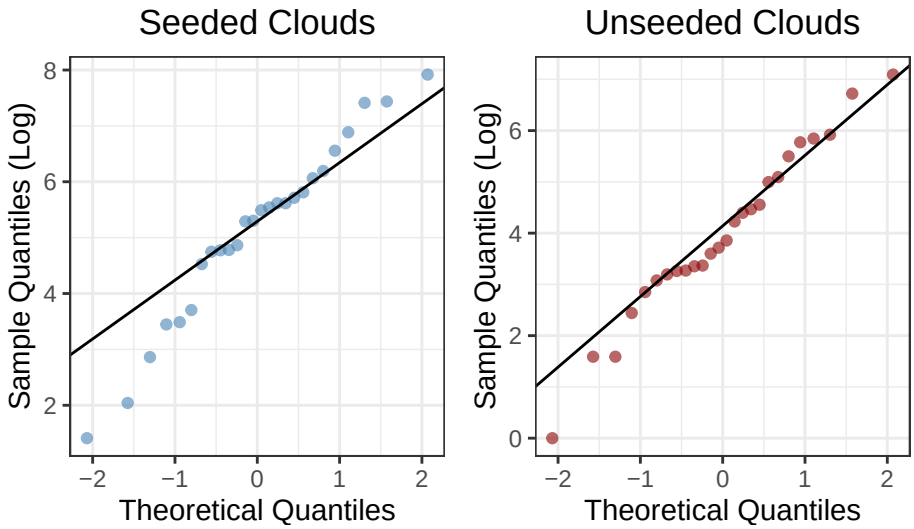

```r
# Save the plot as a PNG
ggsave(filename = "figures/qq_plot_log.png",
  plot = qq_plot_log,
  width = 5,
  height = 3,
  units = "in",
  dpi = 300)
```

QQ Plot, Combined

Before conducting any formal tests, a normal QQ plot was generated to visually assess normality.

```r
# 1. Log-transform the data
log_seeded <- log(cloud$seeded)
log_unseeded <- log(cloud$unseeded)

# 2. Create the Seeded Normal Q-Q Plot
p1 <- ggplot(data.frame(x = log_seeded), aes(sample = x)) +
  stat_qq(color = "steelblue", alpha = 0.6) +
  stat_qq_line(color = "black") +
  labs(title = "Seeded Clouds",
       x = "Theoretical Quantiles",
       y = "Sample Quantiles (Log)") +
  theme_bw() +
  theme(plot.title = element_text(hjust = 0.5))

# 3. Create the Unseeded Normal Q-Q Plot
p2 <- ggplot(data.frame(x = log_unseeded), aes(sample = x)) +
  stat_qq(color = "darkred", alpha = 0.6) +
  stat_qq_line(color = "black") +
  labs(title = "Unseeded Clouds",
       x = "Theoretical Quantiles",
       y = "Sample Quantiles (Log)") +
  theme_bw() +
  theme(plot.title = element_text(hjust = 0.5))
```

```r
# 4. Combine them side-by-side
combined_qq <- p1 + p2
combined_qq
```




```r
# 5. Save the combined plot
ggsave(filename = "figures/qq_plots_combined.png",
       plot = combined_qq,
       width = 8,
       height = 4,
       dpi = 300)
```

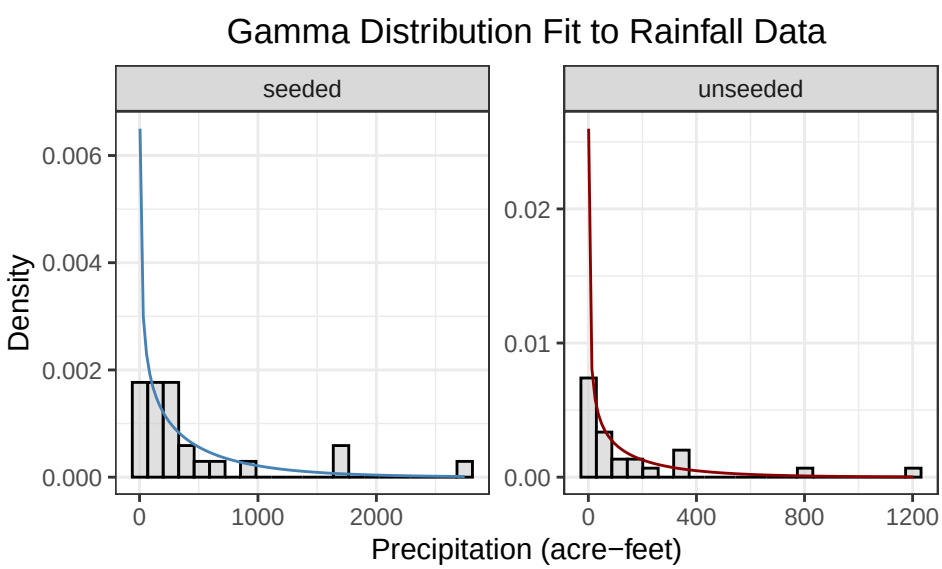
Gamma Fit

```r
fit_seeded = fitdistr(cloud$seeded, "gamma")
fit_unseeded = fitdistr(cloud$unseeded, "gamma")

cat("Gamma Parameters for Seeded Clouds\n")
fit_seeded
cat("Gamma Parameters for Unseeded Clouds\n")
fit_unseeded
```

```
## Gamma Parameters for Seeded Clouds
##         shape              rate
##   0.6395007950345   0.0014468261868
##  (0.1304172358078) (0.0003086768328)
## Gamma Parameters for Unseeded Clouds
##        shape              rate
##   0.560322645687   0.003400926111
##  (0.126803222640) (0.001111210622)
```

```r
shape_s = fit_seeded$estimate["shape"]
rate_s = fit_seeded$estimate["rate"]

shape_u = fit_unseeded$estimate["shape"]
rate_u = fit_unseeded$estimate["rate"]
```

```r
ggplot(cloud_long, aes(x = rain_volume, fill = cloud_type)) +
  geom_histogram(aes(y = after_stat(density)),
                 bins = 22,
                 fill = "gray80",
                 color = "black",
                 alpha = 0.6) +
  stat_function(data = subset(cloud_long, cloud_type == "seeded"),
                fun = dgamma,
                args = list(shape = shape_s, rate = rate_s),
                color = "steelblue",
                linewidth = 0.5) +
  stat_function(data = subset(cloud_long, cloud_type == "unseeded"),
                fun = dgamma,
                args = list(shape = shape_u, rate = rate_u),
                color = "darkred",
                linewidth = 0.5) +
  facet_wrap(~ cloud_type, scales = "free") +
  labs(title = "Gamma Distribution Fit to Rainfall Data",
       x = "Precipitation (acre-feet)",
       y = "Density") +
  theme_bw() +
  theme(plot.title = element_text(hjust = 0.5))
```



Conover Test of Equal Variance

We begin with the Conover test, using a significance level of $\alpha = 0.05$.

Let X denote the precipitation levels for seeded clouds and let Y denote the precipitation levels for unseeded clouds. The assumptions underlying this hypothesis test are:

- Observations $X_1, \ldots, X_m$ are iid from population 1, and $Y_1, \ldots, Y_n$ are iid from population 2,
- The X's and Y's are mutually independent,
- Populations 1 and 2 are continuous.

Although the recorded rainfall measurements are discrete, we conceptualize the underlying process as a continuous latent variable, representing the continuous flow of precipitation. This justifies the continuity assumption. Additionally, ties may occur in the observed data due to discretization, and these are accommodated using

tie-adjusted statistics and asymptotic approximations for the p-value calculation.

Below we quickly check to see if there are any ties present.

```
# Define groups
X = cloud$seeded
Y = cloud$unseeded

# Obtain sample sizes
m = length(X)
n = length(Y)
N = m + n

# Squared deviations from group means
d_X = (X - mean(X))^2
d_Y = (Y - mean(Y))^2

# Combine squared deviations and rank
d_all = c(d_X, d_Y)
ranks = rank(d_all, ties.method = "average")

# Check for ties in the ranked deviations
has_ties = length(unique(ranks)) < length(ranks)

has_ties
```

```
## [1] TRUE
```

The null hypothesis asserts that the variability of the two populations is equal, and the alternative hypothesis asserts that the variances differ. Formally, we have

$$\begin{aligned} H_0: &\quad \sigma_X^2 = \sigma_Y^2, \\ H_A: &\quad \sigma_X^2 \neq \sigma_Y^2. \end{aligned}$$

The Conover procedure involves computing the squared deviations of each observation from its sample mean for both groups. These $m+n$ squared differences are then ranked together, and the sum of the squared ranks for the observations from Y is used to calculate the test statistic.

```
# Split ranks by group
R_X = ranks[1:m]
R_Y = ranks[(m+1):(m+n)]

# Compute T = sum of squared ranks for RV Y
T = sum(R_Y^2)

if (!has_ties) {
  # --- EXACT METHOD ---
  cat("No ties detected. Using Exact Distribution for T\n")
  cat("T:", T, "\n")
  cat("Look up the appropriate percentile of the Conover test")
} else {
  # --- ASYMPTOTIC METHOD ---
  cat("Ties detected. Using Tie-Adjusted Normal Approximation (T*)\n")

  # Tie-adjusted quantities
  V_R = (sum(R_X^2) + sum(R_Y^2)) / N
  W_R = sum(R_X^4) + sum(R_Y^4)
```

```r
  # Standardized test statistic T*
  numerator = T - m * V_R
  denominator = sqrt((m*n/(N*(N-1))) * W_R - (m*n/(N-1)) * V_R^2)
  T_star = numerator / denominator

  # Two-sided p-value
  alpha = 0.05
  crit_val = qnorm(1 - (alpha)/2)

  # Output results
  cat("T:", T, "\n")
  cat("V_R:", V_R, "\n")
  cat("W_R:", W_R, "\n")
  cat("Tie-adjusted statistic (T*):", T_star, "\n")
  cat("Critical Value (z_aplha/2):", crit_val, "\n")
}
```

```
## Ties detected. Using Tie-Adjusted Normal Approximation (T*)
## T: 12326.5
## V_R: 927.4807692
## W_R: 79739412.25
## Tie-adjusted statistic (T*): -3.946119631
## Critical Value (z_aplha/2): 1.959963985
```

For the Conover test, the tie-adjusted test statistic is $T^* = -3.9461$. The associated critical value for this test is $z_{\alpha/2} = 1.960$.

The rejection region is defined as

$$|T^*| \geq z_{\alpha/2}.$$

Since $|T^*| = 3.9461 > 1.960$, the test statistic falls in the rejection region. Therefore, we reject the null hypothesis.

At the $\alpha = 0.05$ significance level, there is sufficient evidence to conclude that the variances of precipitation among the two types of clouds are different. In practical terms, the cloud precipitation do not exhibit similar dispersions across the groups.

Conover Test of Equal Variance, Log Scale

To determine whether a location-shift test is appropriate, we first assess whether the variability of rainfall is similar between seeded and unseeded clouds. We use the Conover test on the log-transformed rainfall data, with a significance level of $\alpha = 0.05$.

Let X denote the log-transformed precipitation levels for seeded clouds and Y denote the log-transformed precipitation levels for unseeded clouds. The assumptions underlying the Conover test are:

- Observations $X_1, \ldots, X_m$ are iid from population 1, and $Y_1, \ldots, Y_n$ are iid from population 2,
- The X's and Y's are mutually independent,
- Populations 1 and 2 are continuous.

Although the recorded rainfall measurements are discrete, we conceptualize the underlying process as a continuous latent variable, representing the continuous flow of precipitation. This justifies the continuity assumption. Additionally, ties may occur in the observed data due to discretization, and these are accommodated using tie-adjusted statistics and asymptotic approximations for the p-value calculation.

Below we quickly check to see if there are any ties present.

```r
# Define log-transformed groups
X = log10(cloud$seeded)
Y = log10(cloud$unseeded)

# Obtain sample sizes
m = length(X)
n = length(Y)
N = m + n

# Squared deviations from group means (on log scale)
d_X = (X - mean(X))^2
d_Y = (Y - mean(Y))^2

# Combine squared deviations and rank
d_all = c(d_X, d_Y)
ranks = rank(d_all, ties.method = "average")

# Check for ties in the ranked deviations
has_ties = length(unique(ranks)) < length(ranks)

has_ties
```

```
## [1] TRUE
```

The null hypothesis asserts that the variability of the two populations is equal, and the alternative hypothesis asserts that the variances differ. Formally, we have

$$\begin{aligned} H_0: &\quad \sigma^2_{\log_{10}(X)} = \sigma^2_{\log_{10}(Y)}, \\ H_A: &\quad \sigma^2_{\log_{10}(X)} \neq \sigma^2_{\log_{10}(Y)}. \end{aligned}$$

The Conover procedure involves computing the squared deviations of each observation from its sample mean for both groups. These $m+n$ squared differences are then ranked together, and the sum of the squared ranks for the observations from Y is used to calculate the test statistic.

```r
# Split ranks by group
R_X = ranks[1:m]
R_Y = ranks[(m+1):(m+n)]

# Compute T = sum of squared ranks for RV Y
T = sum(R_Y^2)

if (!has_ties) {
  # --- EXACT METHOD ---
  cat("No ties detected. Using Exact Distribution for T\n")
  cat("T:", T, "\n")
  cat("Look up the appropriate percentile of the Conover test")
} else {
  # --- ASYMPTOTIC METHOD ---
  cat("Ties detected. Using Tie-Adjusted Normal Approximation (T*)\n")

  # Tie-adjusted quantities
  V_R = (sum(R_X^2) + sum(R_Y^2)) / N
  W_R = sum(R_X^4) + sum(R_Y^4)

  # Standardized test statistic T*
```

```r
  numerator = T - m * V_R
  denominator = sqrt((m*n/(N*(N-1))) * W_R - (m*n/(N-1)) * V_R^2)
  T_star = numerator / denominator

  # Two-sided p-value
  alpha = 0.05
  crit_val = qnorm(1 - (alpha)/2)

  # Output results
  cat("T:", T, "\n")
  cat("V_R:", V_R, "\n")
  cat("W_R:", W_R, "\n")
  cat("Tie-adjusted statistic (T*):", T_star, "\n")
  cat("Critical Value (z_alpha/2):", crit_val, "\n")
}
```

```
## Ties detected. Using Tie-Adjusted Normal Approximation (T*)
## T: 25431.5
## V_R: 927.4807692
## W_R: 79736550.25
## Tie-adjusted statistic (T*): 0.4408934513
## Critical Value (z_alpha/2): 1.959963985
```

For the log-transformed rainfall, the tie-adjusted test statistic is $T^* = 0.4409$. The associated critical value for this test is $z_{\alpha/2} = 1.960$.

The rejection region is defined as

$$|T^*| \geq z_{\alpha/2}.$$

Since $|T^*| = 0.4409 < 1.960$, the test statistic does not fall in the rejection region. Therefore, we fail to reject the null hypothesis of equal dispersions on the log-transformed data.

At the $\alpha = 0.05$ significance level, there is insufficient evidence to reject the null hypothesis of equal dispersions. The log transformation successfully stabilized variances, justifying the use of a location-shift test (such as a Wilcoxon Rank Sum) to compare medians between seeded and unseeded clouds.

Wilcoxon Sum Rank Test

After verifying that the log-transformed rainfall distributions have similar dispersion using the Conover test, we proceed with the Wilcoxon Rank Sum test to compare the central location of the two populations.

Let X denote the log-transformed precipitation measurements for seeded cloud and Y denote the log-transformed precipitation measurements for unseeded clouds. To be consistent with the textbook notation, define

$$\Delta = \theta_X - \theta_Y,$$

where θ represents the location parameter (median).

The assumptions underlying this hypothesis test are:

- Observations $X_1, \ldots, X_m$ are iid from population 1, and $Y_1, \ldots, Y_n$ are iid from population 2,
- The X's and Y's are mutually independent,
- Populations 1 and 2 are continuous,
- Under the null hypothesis, the two populations have the same distributional shape and spread. This assumption is supported by the Conover test, which found no evidence of unequal dispersion.

We therefore conduct the following one-sided hypothesis test

$$\begin{aligned} H_0: &\quad \Delta \leq 0, \\ H_A: &\quad \Delta > 0. \end{aligned}$$

This hypothesis test is performed below.

```r
# Wilcoxon Rank Sum test
# Documentation states that it uses x-y,
wilcox.exact(x = log10(cloud$seeded),
             y = log10(cloud$unseeded),
             paired = FALSE,
             alternative = "greater",
             exact = TRUE,
             conf.level = 0.95,
             conf.int = TRUE)
```

```
##
##  Exact Wilcoxon rank sum test
##
## data:  log10(cloud$seeded) and log10(cloud$unseeded)
## W = 473, p-value = 0.006390019
## alternative hypothesis: true mu is greater than 0
## 95 percent confidence interval:
##  0.1634287154          Inf
## sample estimates:
## difference in location
##           0.5469905318
```

The function `wilcox.exact()` reports the Mann-Whitney U statistic. To align with the Wilcoxon Rank Sum tables, we convert U to the Wilcoxon statistic W using

$$\begin{aligned} W &= U + \frac{n(n+1)}{2} \\ &= 473 + \frac{26 \cdot 27}{2} \\ &= 824. \end{aligned}$$

Because the sample sizes are moderately large ($m = n = 26$), the distribution of the Wilcoxon statistic can be approximated using a normal distribution. The corresponding standardized statistic is

$$\begin{aligned} W^* &= \frac{W - \frac{n(m+n+1)}{2}}{\sqrt{\frac{mn(m+n+1)}{12}}} \\ &= \frac{824 - \frac{26(53)}{2}}{\sqrt{\frac{676(53)}{12}}} \\ &= \frac{135}{\sqrt{\frac{8957}{3}}} \\ &= 2.471. \end{aligned}$$

For a one-sided test at significance level $\alpha = 0.05$, the critical value from the standard normal distribution is

$$z_{0.05} = 1.6449.$$

```r
qnorm(1 - 0.05)
```

```
## [1] 1.644853627
```

The rejection region is given as

$$W^* \geq z_{0.05}.$$

Since $W^* = 2.471 > 1.6449$, we reject the null hypothesis. At the 5% significance level, there is sufficient statistical evidence to conclude that seeded clouds tend to produce larger precipitation amounts than unseeded clouds on the log scale.

Hodges-Lehmann estimator

```r
# Log-transformed rainfall for seeded and unseeded clouds
log_seeded   <- log10(cloud$seeded)
log_unseeded <- log10(cloud$unseeded)

# Compute all pairwise differences: seeded - unseeded
diffs <- outer(log_seeded, log_unseeded, "-")

# Hodges-Lehmann estimator: median of all pairwise differences
hl_manual <- median(diffs)
hl_manual
```

```
## [1] 0.5469905318
```

```r
# Convert back to original units (multiplicative effect)
k <- 10^hl_manual
k
```

```
## [1] 3.52363189
```